

Quantum Computation

Module 2 Assignment

*Single-Qubit State Vectors, Pauli Gates and the Hadamard Gate in Qiskit*

Albert Kabore

Morgan State University

COSC 586 Quantum Computing

Dr. Eric Sakk

October 5, 2026

**Question 1 - Showing that  $|\psi\rangle$  is a valid state vector**

The question gives  $|\psi\rangle = (i/2)|0\rangle + (\sqrt{3} e^{i\pi/6}/2)|1\rangle$ . Use the computational basis  $|0\rangle = (1, 0)^T$  and  $|1\rangle = (0, 1)^T$ . These vectors are orthonormal:  $\langle 0|0\rangle = \langle 1|1\rangle = 1$  and  $\langle 0|1\rangle = \langle 1|0\rangle = 0$ .

The conjugate transpose is  $\langle\psi| = (-i/2)\langle 0| + (\sqrt{3}/2)e^{i\pi/6}\langle 1|$ .

Expanding  $\langle\psi|\psi\rangle$  gives  $|a_0|^2\langle 0|0\rangle + a_0^* a_1\langle 0|1\rangle + a_1^* a_0\langle 1|0\rangle + |a_1|^2\langle 1|1\rangle$ . The two cross terms vanish because the basis vectors are orthogonal.

$|a_0|^2 = (-i/2)(i/2) = -i^2/4 = 1/4$ . Also,  $|a_1|^2 = (\sqrt{3}/2)^2 e^{i\pi/6}e^{-i\pi/6} = (3/4)e^0 = 3/4$ . Thus  $\langle\psi|\psi\rangle = 1/4 + 3/4 = 1$  and  $\|\psi\| = \sqrt{1} = 1$ . This proves the required normalization without treating complex amplitudes as ordinary real squares.

As an independent algebra check,  $e^{i\pi/6} = \sqrt{3}/2 + i/2$ , so  $a_1 = 3/4 + i\sqrt{3}/4$ . Its squared magnitude is  $(3/4)^2 + (\sqrt{3}/4)^2 = 9/16 + 3/16 = 3/4$ .

### Question

Given the single-qubit state vector  $|\psi\rangle = a_0|0\rangle + a_1|1\rangle$ , show that the following ket is a valid state vector (i.e. its magnitude is normalised to one):  $|\psi\rangle = \frac{i}{2}|0\rangle + \frac{\sqrt{3}e^{i\pi/6}}{2}|1\rangle$

### Step 1 – Identify the amplitudes

For  $|\psi\rangle = a_0|0\rangle + a_1|1\rangle$ , the amplitudes are:

$$a_0 = \frac{i}{2}, \quad a_1 = \frac{\sqrt{3}e^{i\pi/6}}{2}$$

A valid normalized state must satisfy  $|a_0|^2 + |a_1|^2 = 1$ .

### Step 2 – Squared magnitude of $a_0$

Multiply the amplitude by its complex conjugate:

$$|a_0|^2 = a_0 a_0^* = \left(\frac{i}{2}\right)\left(\frac{-i}{2}\right) = \frac{-i^2}{4} = \frac{-(-1)}{4} = \frac{1}{4}$$

### Step 3 – Squared magnitude of $a_1$

The phase has magnitude one, so:

$$|a_1|^2 = \left( \frac{\sqrt{3} e^{i\pi/6}}{2} \right) \left( \frac{\sqrt{3} e^{-i\pi/6}}{2} \right) = \frac{3}{4} e^{i\pi/6 - i\pi/6} = \frac{3}{4} e^0 = \frac{3}{4}$$

### Step 4 - Normalization

$$\langle \psi | \psi \rangle = |a_0|^2 + |a_1|^2 = \frac{1}{4} + \frac{3}{4} = 1$$

Thus the norm is the square root of 1, which is 1. The given ket is a valid normalized state vector.

### Conclusion for Question 1

Since  $\langle \psi | \psi \rangle = 1$ , the magnitude of  $|\psi\rangle$  is normalised to one, so it is a valid single-qubit state vector. Physically, a measurement would give 0 with probability  $P(|0\rangle) = \frac{1}{4} = 25\%$  and 1 with probability  $P(|1\rangle) = \frac{3}{4} = 75\%$ . Note that the phases  $i$  and  $e^{i\pi/6}$  do not change these probabilities; only the magnitudes matter.

**Question 1 - Normalisation**

For  $|\psi\rangle = \frac{i}{2}|0\rangle + \frac{\sqrt{3}e^{i\pi/6}}{2}|1\rangle$ , the orthonormal basis gives

$$\langle\psi|\psi\rangle = |a_0|^2 + |a_1|^2 = \frac{(-i)i}{4} + \frac{3e^{-i\pi/6}e^{i\pi/6}}{4} = \frac{1}{4} + \frac{3}{4} = 1.$$

Therefore  $\|\psi\| = \sqrt{1} = 1$ , so the ket is a valid normalized state vector. The following cell checks the calculation numerically.

```

a0 = 1j / 2
a1 = np.sqrt(3) * np.exp(1j * np.pi / 6) / 2

print(f"a0 = {a0:.4f}")
print(f"a1 = {a1:.4f}    (= 3/4 + i*sqrt(3)/4 in Cartesian form)")
print(f"|a0|^2 = {abs(a0)**2:.4f}")
print(f"|a1|^2 = {abs(a1)**2:.4f}")
norm_sq = abs(a0)**2 + abs(a1)**2
print(f"|a0|^2 + |a1|^2 = {norm_sq:.4f}")

psi = Statevector([a0, a1])
print("Qiskit Statevector.is_valid():", psi.is_valid())
assert np.isclose(norm_sq, 1.0)

```

```

[2]

... a0 = 0.0000+0.5000j
a1 = 0.7500+0.4330j    (= 3/4 + i*sqrt(3)/4 in Cartesian form)
|a0|^2 = 0.2500
|a1|^2 = 0.7500
|a0|^2 + |a1|^2 = 1.0000
Qiskit Statevector.is_valid(): True

```

## Question 2 - $H|0\rangle$ , $Y|0\rangle$ and $Z|0\rangle$ in Qiskit

The three requested inputs are  $|0\rangle$ , independently of the state in Question 1.  $H$  is the Hadamard gate;  $Y$  and  $Z$  are Pauli gates. Although the instructions recall  $X$ , they request implementations only of  $H|0\rangle$ ,  $Y|0\rangle$  and  $Z|0\rangle$ .

For an output  $b_0|0\rangle + b_1|1\rangle$ , computational-basis measurement obeys the Born rule:  $P(|0\rangle) = |b_0|^2$  and  $P(|1\rangle) = |b_1|^2$ . Derive these amplitudes by matrix multiplication before comparing them with simulator output. State vectors are computed before measurement; the measured circuit then produces classical counts.

## Question

Using `qc = QuantumCircuit(1,1)` and the gate syntax `qc.x(0)`, `qc.y(0)`, `qc.z(0)` (and `qc.h(0)`), implement (a)  $H|0\rangle$ , (b)  $Y|0\rangle$ , (c)  $Z|0\rangle$ . For each: (i) measure and show the histogram, (ii) show the circuit drawn with `draw()`, (iii) derive the output state and  $P(|0\rangle)$ ,  $P(|1\rangle)$  and verify them against the histogram, (iv) put everything in one document.

| Gate | Qiskit call          | Action on $ 0\rangle$                    | Action on $ 1\rangle$                    | Intuition                   |
|------|----------------------|------------------------------------------|------------------------------------------|-----------------------------|
| $X$  | <code>qc.x(0)</code> | $ 1\rangle$                              | $ 0\rangle$                              | Bit flip (quantum NOT)      |
| $Y$  | <code>qc.y(0)</code> | $i 1\rangle$                             | $-i 0\rangle$                            | Bit flip + phase flip       |
| $Z$  | <code>qc.z(0)</code> | $ 0\rangle$                              | $- 1\rangle$                             | Phase flip                  |
| $H$  | <code>qc.h(0)</code> | $\frac{ 0\rangle +  1\rangle}{\sqrt{2}}$ | $\frac{ 0\rangle -  1\rangle}{\sqrt{2}}$ | Creates equal superposition |

## Implementation

Each circuit uses one qubit initialized in  $|0\rangle$  and one classical bit. The notebook implements the following steps:

1. `qc = QuantumCircuit(1, 1)` creates 1 qubit and 1 classical bit. These ideal simulations start the qubit in  $|0\rangle$ .
2. The gate is applied, e.g. `qc.h(0)` acts on qubit 0.
3. `Statevector.from_instruction(qc)` gives the exact output state, used to double-check the hand-derived prediction.

4. `qc.measure(0, 0)` stores the measurement result in the classical bit. The circuit is drawn with `qc.draw()` and run 1024 times (*shots*) on the AerSimulator (seed 42 for reproducibility). `plot_histogram` then shows how often each outcome occurred.

Each shot starts from a fresh  $|0\rangle$  state. The histograms show counts; dividing by 1024 gives the observed frequencies. These simulations use no noise model.

### (a) $H|0\rangle$

Detailed hand calculation:  $H = (1/\sqrt{2})[[1, 1], [1, -1]]$ . Multiplying by  $(1, 0)^T$  gives  $(1/\sqrt{2})(1 \cdot 1 + 1 \cdot 0, 1 \cdot 1 + (-1) \cdot 0)^T = (1/\sqrt{2}, 1/\sqrt{2})^T$ . Hence  $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ .

Both amplitudes are  $1/\sqrt{2}$ , so  $P(|0\rangle) = (1/\sqrt{2})^2 = 1/2$  and  $P(|1\rangle) = 1/2$ . Their sum is 1. For 1024 shots the expected counts are 512 and 512; expected counts are averages over repeated experiments, not a requirement that every run gives exactly these counts.

### Qiskit code

```
qc = QuantumCircuit(1, 1)

qc.h(0)

state = Statevector.from_instruction(qc)

qc.measure(0, 0)

print(qc.draw())

sim = AerSimulator()

raw = sim.run(transpile(qc, sim, seed_transpiler=42), shots=1024,
seed_simulator=42).result().get_counts()

counts = {k: raw.get(k, 0) for k in ("0", "1")}

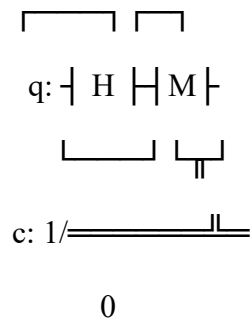
print("Statevector:", np.round(state.data, 4))
```

```
print("Counts:", counts)

print("Observed frequencies:", {k: v / 1024 for k, v in counts.items()})

display(plot_histogram(counts))
```

Output (the generated histogram appears below with its mathematical verification):



Quantum circuit diagram showing a qubit  $q$  initialized to  $|0\rangle$ , followed by a Hadamard gate ( $H$ ), then a multi-controlled NOT gate with two controls ( $c[0]$  and  $c[1]$ ) and a target qubit  $q$ . Finally, another Hadamard gate ( $H$ ) is applied to  $q$ .

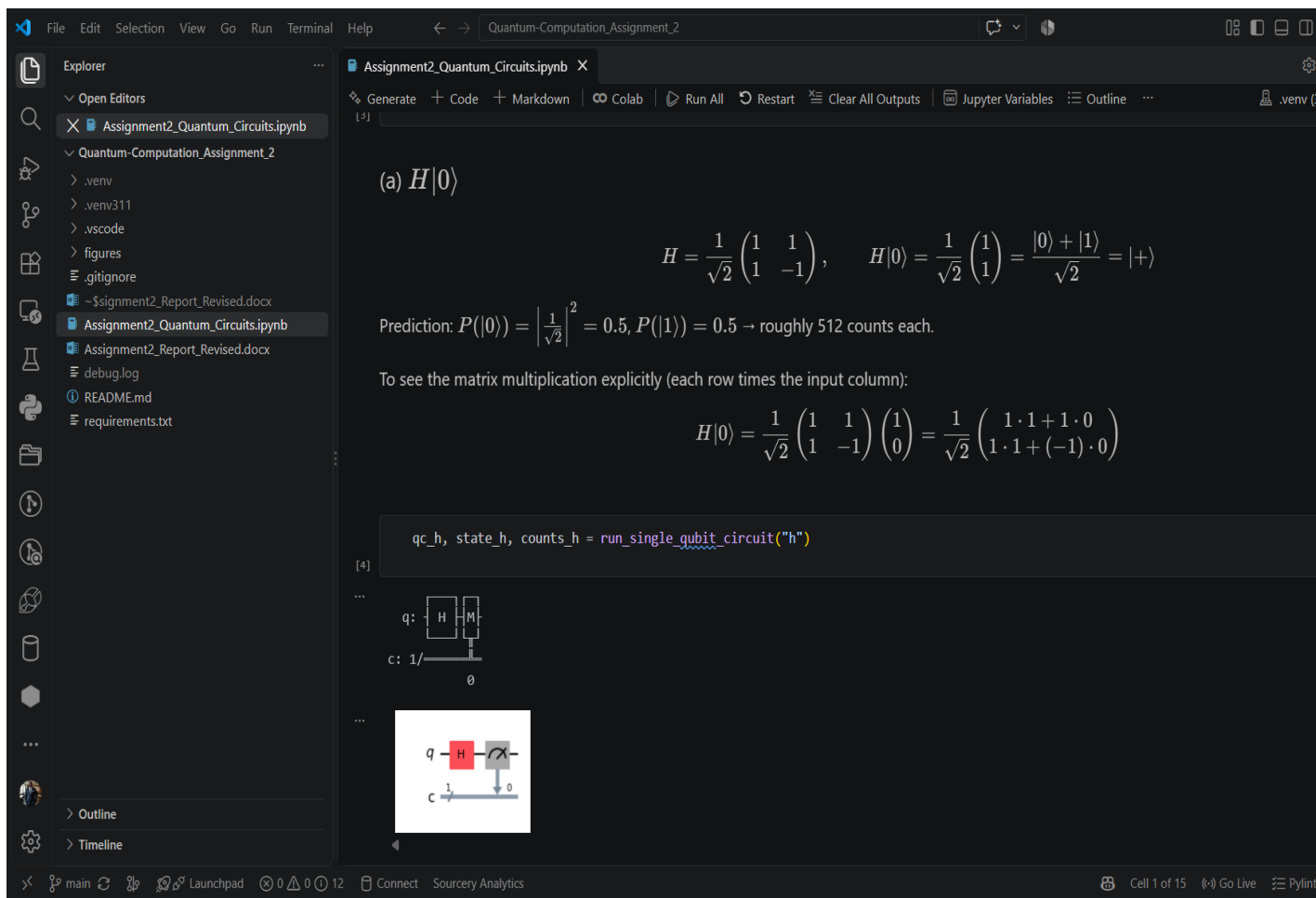
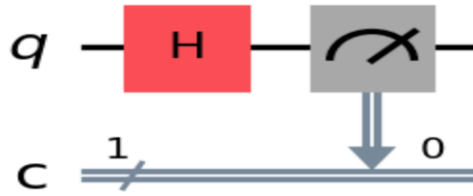
Statevector:  $[0.7071+0.j \ 0.7071+0.j]$

Counts:  $\{'0': 526, '1': 498\}$

Observed frequencies:  $\{'0': 0.513671875, '1': 0.486328125\}$

Figure(640x480)

## (ii) Circuit after running draw()



**Figure 1a - Screenshot of the executed notebook circuit output for  $H|0\rangle$ , drawn with `qc.draw("mpl")`, including measurement into classical bit 0.**

## (iii) Mathematical prediction

Using  $|0\rangle = (1, 0)^T$ , multiply by the Hadamard matrix:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot 1 + 1 \cdot 0 \\ 1 \cdot 1 + (-1) \cdot 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Writing the column vector back in ket form:

$$H|0\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = |+\rangle$$

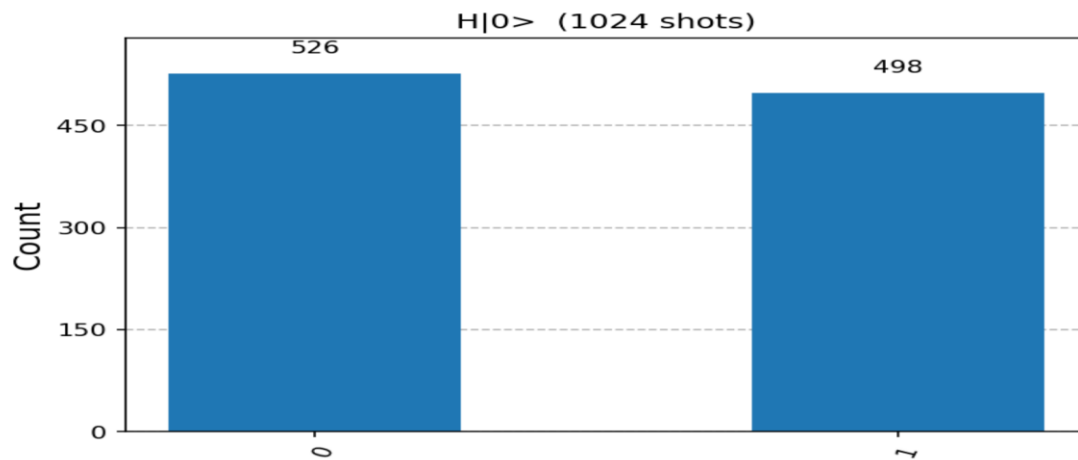
This is the equal superposition state, often called  $|+\rangle$ . Qiskit's Statevector gives  $[0.7071+0.j, 0.7071+0.j]$ , and  $\frac{1}{\sqrt{2}} \approx 0.7071$  ✓.

### Predicted probabilities

$$P(|0\rangle) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 0.5, \quad P(|1\rangle) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 0.5$$

Check:  $0.5 + 0.5 = 1$  ✓. Expected counts for 1024 shots:  $1024 \times 0.5 = 512$  for each outcome.

## (i) Measurement histogram



Quantum-Computation\_Assignment\_2

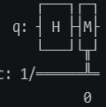
Assignment2\_Quantum\_Circuits.ipynb

Generate + Code + Markdown Colab Run All Restart Clear All Outputs Jupyter Variables Outline .venv (3)

```
qc_h, state_h, counts_h = run_single_qubit_circuit("h")
```

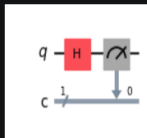
[4]

...

q: 

c: 1/0

...



...

H|0> (1024 shots)

| Measurement Outcome | Count |
|---------------------|-------|
| 0                   | 526   |
| 1                   | 498   |

...

Statevector: [0.7071+0.j 0.7071+0.j]  
Predicted P(0) = 0.5000 P(1) = 0.5000  
Observed f(0) = 0.5137 f(1) = 0.4863 counts = {'0': 526, '1': 498}

main Launchpad 0 0 0 1 Connect Sourcery Analytics Cell 1 of 15 Go Live Pylint

**Figure 1b - Screenshot of the executed notebook measurement histogram for  $H|0\rangle$ , 1024 shots: 526 zeros and 498 ones.**

### Verification

The histogram shows 526 zeros and 498 ones, i.e. relative frequencies

$$\hat{P}(|0\rangle) = \frac{526}{1024} \approx 0.514, \quad \hat{P}(|1\rangle) = \frac{498}{1024} \approx 0.486$$

The observed frequencies  $526/1024 = 0.5137$  and  $498/1024 = 0.4863$  are close to the predicted 0.5 and 0.5. The small difference is consistent with finite sampling.

(b)  $Y|0\rangle$

Detailed hand calculation:  $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$ . Therefore  $Y(1, 0)^T = (0 \cdot 1 + (-i) \cdot 0, i \cdot 1 + 0 \cdot 0)^T = (0, i)^T = i|1\rangle$ .

The output amplitudes are  $b_0 = 0$  and  $b_1 = i$ . Thus  $P(|0\rangle) = 0$  and  $P(|1\rangle) = i^* i = (-i)i = 1$ . The factor  $i$  multiplies the entire state by a global phase, so it does not change the outcome probabilities. Every ideal shot yields 1.

### Qiskit code

```
qc = QuantumCircuit(1, 1)
qc.y(0)
state = Statevector.from_instruction(qc)
qc.measure(0, 0)
print(qc.draw())
sim = AerSimulator()
raw = sim.run(transpile(qc, sim, seed_transpiler=42), shots=1024,
```

```

seed_simulator=42).result().get_counts()

counts = {k: raw.get(k, 0) for k in ("0", "1")}

print("Statevector:", np.round(state.data, 4))

print("Counts:", counts)

print("Observed frequencies:", {k: v / 1024 for k, v in counts.items()})

display(plot_histogram(counts))

```

Output (the generated histogram appears below with its mathematical verification):

q:  $\frac{1}{\sqrt{2}}$  Y  $\frac{1}{\sqrt{2}}$  M  $\frac{1}{\sqrt{2}}$

c:  $\frac{1}{\sqrt{2}}$

0

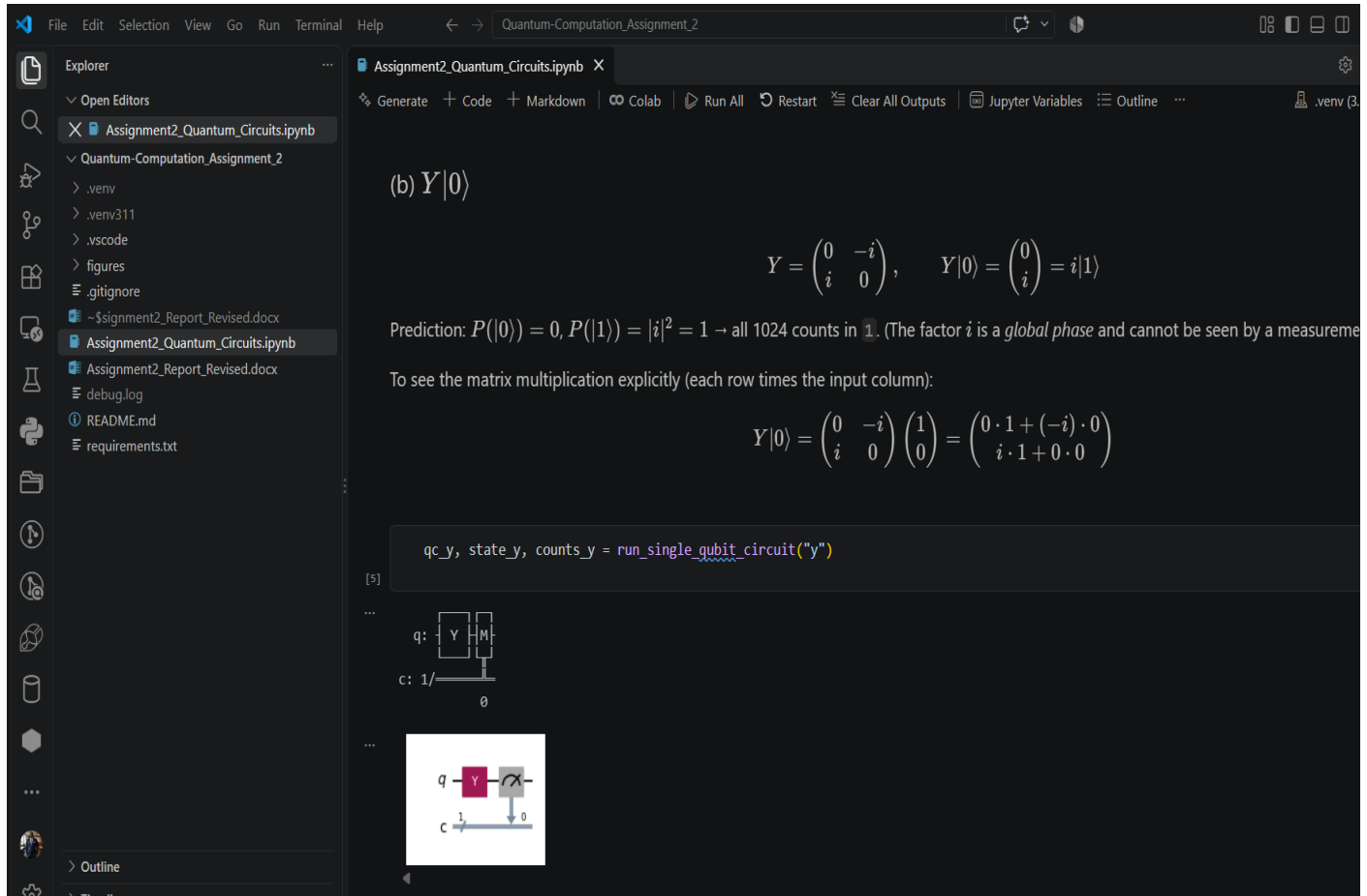
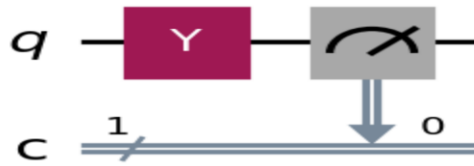
Statevector: [0.+0.j 0.+1.j]

Counts: {'0': 0, '1': 1024}

Observed frequencies: {'0': 0.0, '1': 1.0}

Figure(640x480)

**(ii) Circuit after running draw()**



**Figure 2a - Screenshot of the executed notebook circuit output for  $Y|0\rangle$ , drawn with `qc.draw("mpl")`, including measurement into classical bit 0.**

### (iii) Mathematical prediction

Multiply  $|0\rangle$  by the Pauli-Y matrix:

$$Y|0\rangle = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 + (-i) \cdot 0 \\ i \cdot 1 + 0 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0 \\ i \end{pmatrix}$$

In ket form:

$$Y|0\rangle = 0|0\rangle + i|1\rangle = i|1\rangle$$

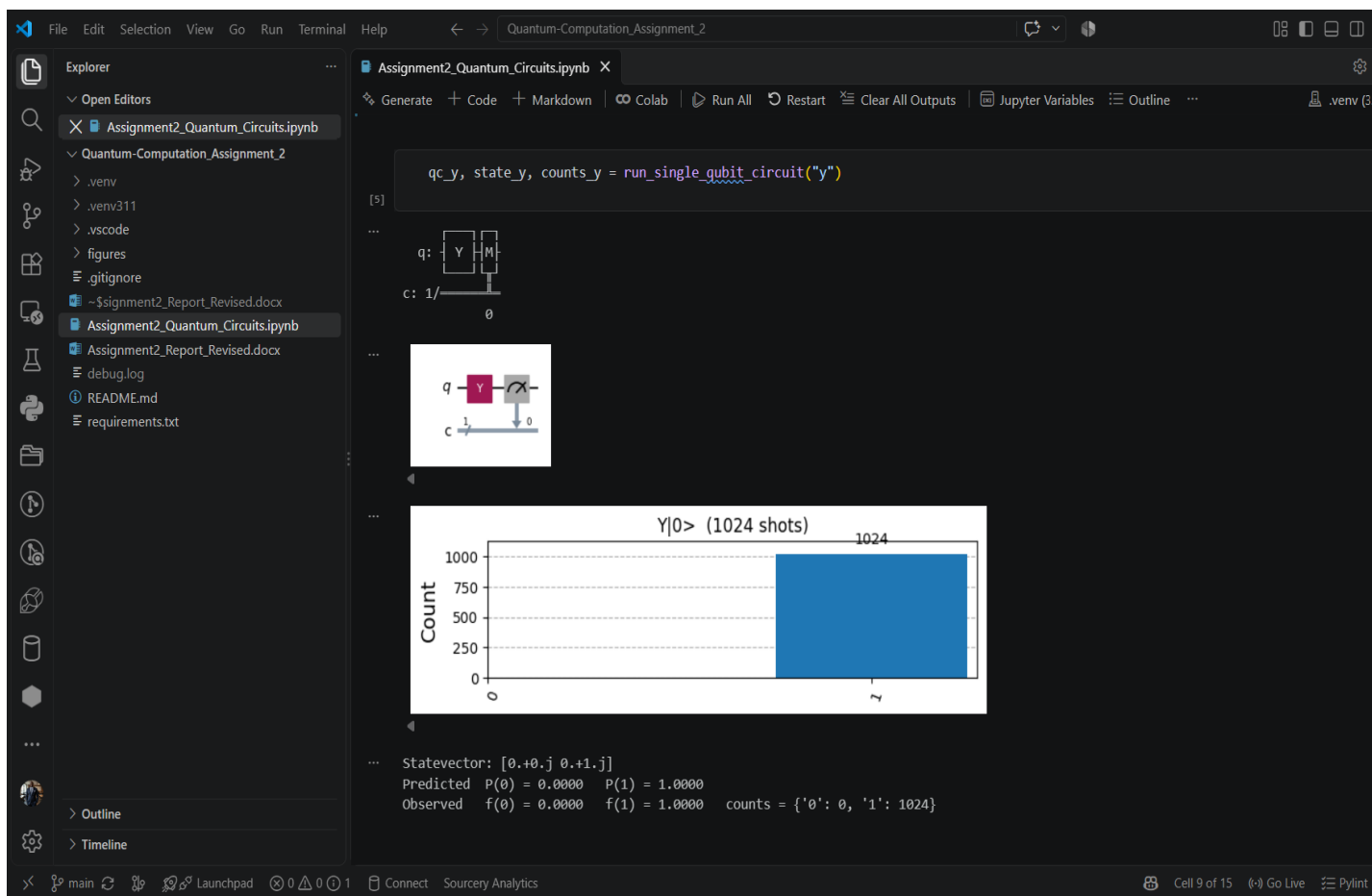
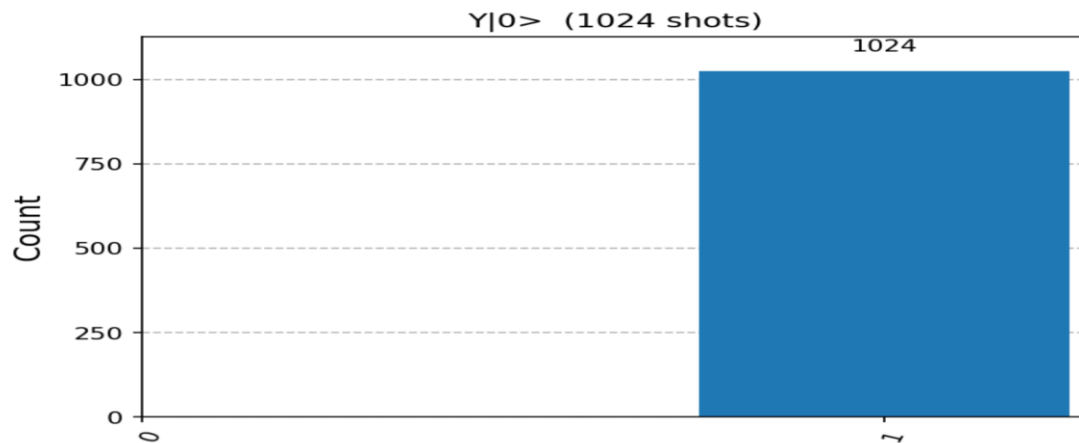
Qiskit gives the state vector  $(0, i)^T$ , agreeing with the derivation. The factor  $i$  is a global phase and does not change the measurement probabilities.

### **Predicted probabilities**

$$P(|0\rangle) = |0|^2 = 0, \quad P(|1\rangle) = |i|^2 = i \cdot (-i) = -i^2 = 1$$

Check:  $0 + 1 = 1$  ✓. Expected counts: 0 for '0' and all 1024 for '1'.

## (i) Measurement histogram



**Figure 2b - Screenshot of the executed notebook measurement histogram for  $Y|0\rangle$ , 1024 shots: 0 zeros and 1024 ones.**

## Verification

The histogram shows **1024 counts of '1' and 0 counts of '0'**, so  $\hat{P}(|1\rangle) = 1024/1024 = 1$  and  $\hat{P}(|0\rangle) = 0$ . This matches the prediction exactly. In this ideal noiseless simulation, a probability of 1 is deterministic, so there is no statistical spread.

Why does the factor  $i$  not show up in the histogram?

$Y|0\rangle = i|1\rangle$  differs from  $|1\rangle$  by a global phase. The squared magnitude is  $|i|^2 = (-i)i = 1$ , so computational-basis measurement gives outcome 1 with certainty.

### (c) $Z|0\rangle$

Detailed hand calculation:  $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ . Therefore  $Z(1, 0)^T = (1 \cdot 1 + 0 \cdot 0, 0 \cdot 1 + (-1) \cdot 0)^T = (1, 0)^T = |0\rangle$ .

The output amplitudes are  $b_0 = 1$  and  $b_1 = 0$ , so  $P(|0\rangle) = |1|^2 = 1$  and  $P(|1\rangle) = |0|^2 = 0$ .  $Z$  changes the sign of a  $|1\rangle$  component, but the requested input has no such component. Every ideal shot yields 0.

## Qiskit code

```
qc = QuantumCircuit(1, 1)

qc.z(0)

state = Statevector.from_instruction(qc)

qc.measure(0, 0)

print(qc.draw())

sim = AerSimulator()

raw = sim.run(transpile(qc, sim, seed_transpiler=42), shots=1024,
seed_simulator=42).result().get_counts()
```

```

counts = {k: raw.get(k, 0) for k in ("0", "1")}

print("Statevector:", np.round(state.data, 4))

print("Counts:", counts)

print("Observed frequencies:", {k: v / 1024 for k, v in counts.items()})

display(plot_histogram(counts))

```

Output (the generated histogram appears below with its mathematical verification):

$$\begin{array}{c}
 \overline{\phantom{0}} \quad \overline{\phantom{0}} \\
 q: \mid Z \mid M \mid \\
 \phantom{q:} \overline{\phantom{0}} \quad \overline{\phantom{0}} \\
 c: 1/\overline{\phantom{0}} \overline{\phantom{0}} \\
 0
 \end{array}$$

Statevector: [1.+0.j 0.+0.j]

Counts: {'0': 1024, '1': 0}

Observed frequencies: {'0': 1.0, '1': 0.0}

Figure(640x480)

## (ii) Circuit after running draw()



Assignment2\_Quantum\_Circuits.ipynb

Quantum-Computation\_Assignment\_2

Quantum-Computation\_Assignment\_2

Assignment2\_Quantum\_Circuits.ipynb

Assignment2\_Report\_Revised.docx

debug.log

README.md

requirements.txt

(c)  $Z|0\rangle$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$$

Prediction:  $P(|0\rangle) = 1, P(|1\rangle) = 0 \rightarrow$  all 1024 counts in 0.

To see the matrix multiplication explicitly (each row times the input column):

$$Z|0\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + (-1) \cdot 0 \end{pmatrix}$$

$Z$  leaves the input  $|0\rangle$  unchanged.

```
qc_z, state_z, counts_z = run_single_qubit_circuit("z")
```

[6]

...

q:

c: 1/0

0

...

Spaces: 4 { } Cell 10 of 15 Go Live Pylint

**Figure 3a - Screenshot of the executed notebook circuit output for  $Z|0\rangle$ , drawn with `qc.draw("mpl")`, including measurement into classical bit 0.**

### **(iii) Mathematical prediction**

Multiply  $|0\rangle$  by the Pauli-Z matrix:

$$Z|0\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + (-1) \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

In ket form:

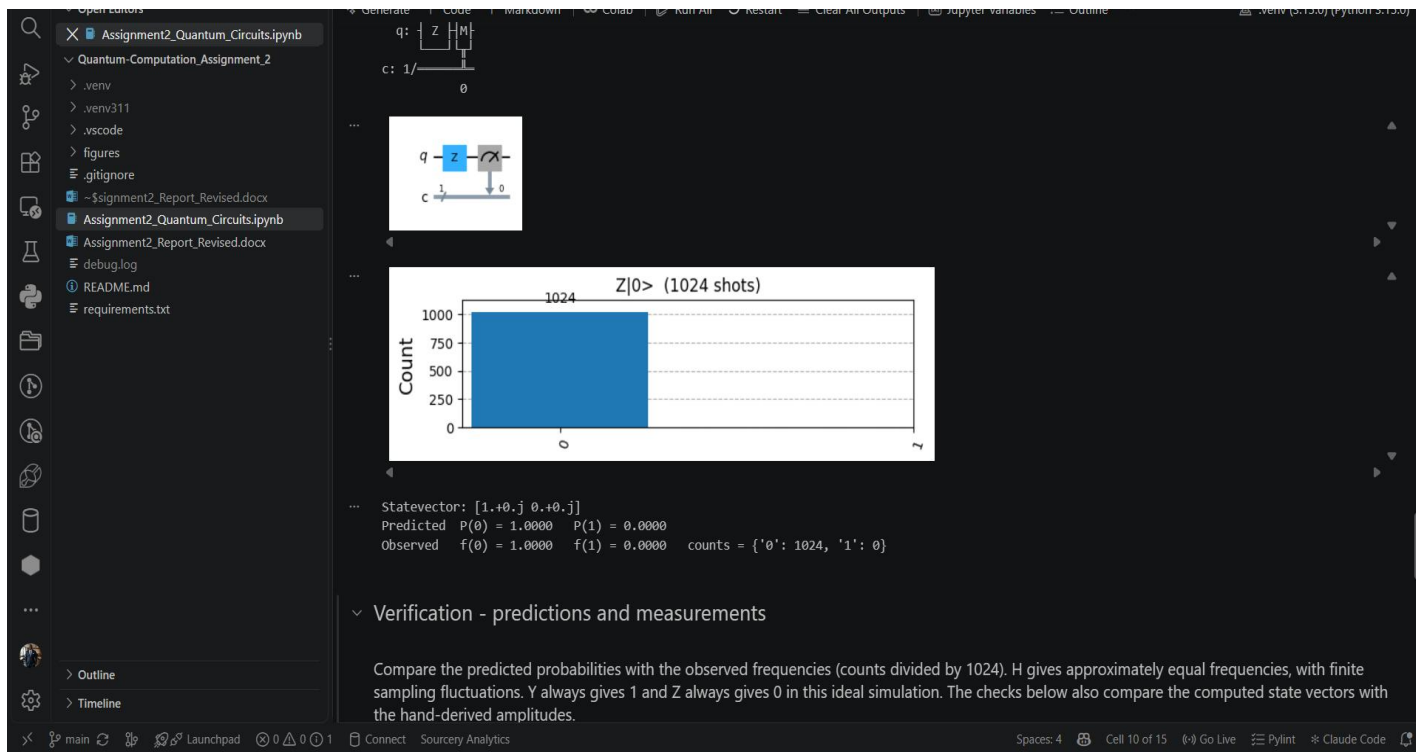
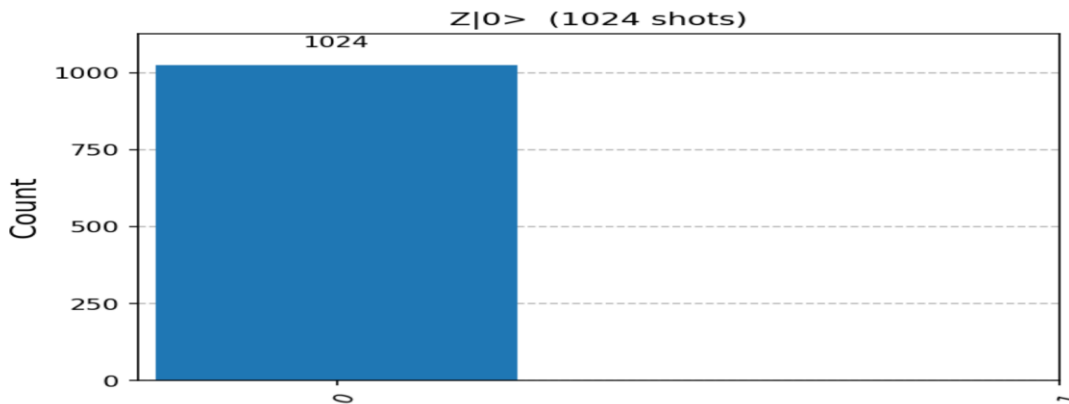
$$Z|0\rangle = |0\rangle$$

Qiskit gives the state vector  $(1, 0)^T$ , agreeing with the derivation.

### **Predicted probabilities**

$$P(|0\rangle) = |1|^2 = 1, \quad P(|1\rangle) = |0|^2 = 0$$

Check:  $1 + 0 = 1$  ✓. Expected counts: all 1024 for '0' and 0 for '1'.(i) Measurement histogram



**Figure 3b - Screenshot of the executed notebook measurement histogram for  $Z|0\rangle$ , 1024 shots: 1024 zeros and 0 ones.**

## Verification

The histogram shows **1024 counts of '0' and 0 counts of '1'**, so  $\hat{P}(|0\rangle) = 1$  and  $\hat{P}(|1\rangle) = 0$ , in exact agreement with the prediction.

### What does Z actually do?

Z is a **phase flip**: it multiplies the  $|1\rangle$  amplitude by  $-1$  and leaves the  $|0\rangle$  amplitude alone,  $Z(a_0|0\rangle + a_1|1\rangle) = a_0|0\rangle - a_1|1\rangle$ . Starting from  $|0\rangle$  there is no  $|1\rangle$  component to flip, so nothing changes. Its effect is visible only on superpositions, e.g.  $Z|+\rangle = |-\rangle$ , and even then only after a change of measurement basis.

### Comparison of predictions and measurements

The H run gives frequencies  $526/1024 = 0.513671875$  and  $498/1024 = 0.486328125$ . Their deviations from 0.5 are  $+0.013671875$  and  $-0.013671875$ . For independent shots with  $p = 0.5$ , the standard deviation of the count of zeros is  $\sqrt{(1024 \times 0.5 \times 0.5)} = 16$ . The observed difference of 14 counts from 512 is 0.875 standard deviations, consistent with finite sampling. This comparison supports the prediction; the exact state-vector check independently verifies the amplitudes.

Y gives  $0/1024 = 0$  for outcome 0 and  $1024/1024 = 1$  for outcome 1. Z gives  $1024/1024 = 1$  for outcome 0 and  $0/1024 = 0$  for outcome 1. Both match the predictions exactly. These are ideal AerSimulator results, not measurements from a physical quantum processor.

The table collects the hand-derived predictions and the measured results (1024 shots each).

| Circuit | Predicted output<br>state | Predicted<br>P(0), P(1) | Measured<br>counts '0' /<br>'1' | Observed<br>f(0), f(1) | Agree? |
|---------|---------------------------|-------------------------|---------------------------------|------------------------|--------|
|---------|---------------------------|-------------------------|---------------------------------|------------------------|--------|

|              |                                             |          |           |              |                         |
|--------------|---------------------------------------------|----------|-----------|--------------|-------------------------|
| $H 0\rangle$ | $\frac{1}{\sqrt{2}}( 0\rangle +  1\rangle)$ | 0.5, 0.5 | 526 / 498 | 0.514, 0.486 | Yes (within $1\sigma$ ) |
| $Y 0\rangle$ | $i 1\rangle$                                | 0, 1     | 0 / 1024  | 0, 1         | Yes (exact)             |
| $Z 0\rangle$ | $ 0\rangle$                                 | 1, 0     | 1024 / 0  | 1, 0         | Yes (exact)             |

## References

- IBM Quantum Learning – Basics of Quantum Information, “Single systems”.

<https://learning.quantum.ibm.com/course/basics-of-quantum-information/single-systems>